

SOFTWARE ENGINEERING COURSE

LECTURE 7

Software Testing

Dr. Micheal Tuape



MODULE 2 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 9	Oct. 28–Nov1, 2024	Software Testing & Quality Management	Chapter 8 & 24
Week 10	Nov. 4– Nov 8, 2024	Software Evolution & Configuration Management	Chapter 9 & 25
Week 11	Nov. 11– Nov 15, 2024	Software Project Management & Software Project Planning	None
Week 12	Nov. 18– Nov 22, 2024	Software Quality and Configuration	Chapter 22 & 23
Week 13	Nov. 25– Nov 29, 2024	Guest Lecture: Customer Success & Software Development	None
Week 14	December 2–6, 2023	Trends in Software Engineering	None





LEARNING OBJECTIVES

- »» Development testing
- »» Test-driven development
- »» Release testing
- »» User testing





PROGRAM TESTING

- »» Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- »» When you test software, you execute a program using artificial data.
- »» You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- »» Can reveal the presence of errors not their absence.
- »» Testing is part of a more general verification and validation process, which also includes static validation techniques.





PROGRAM TESTING GOALS

- »» To demonstrate to the developer and the customer that the software meets its requirements.
 - For custom software, this means that there should be at least one test for every requirement in the requirements document. For generic software products, it means that there should be tests for all of the system features, plus combinations of these features, that will be incorporated in the product release.
- »» To discover situations in which the behavior of the software is incorrect, undesirable or does not conform to its specification.
 - Defect testing is concerned with rooting out undesirable system behavior such as system crashes, unwanted interactions with other systems, incorrect computations and data corruption.





VALIDATION AND DEFECT TESTING

»» The first goal leads to validation testing

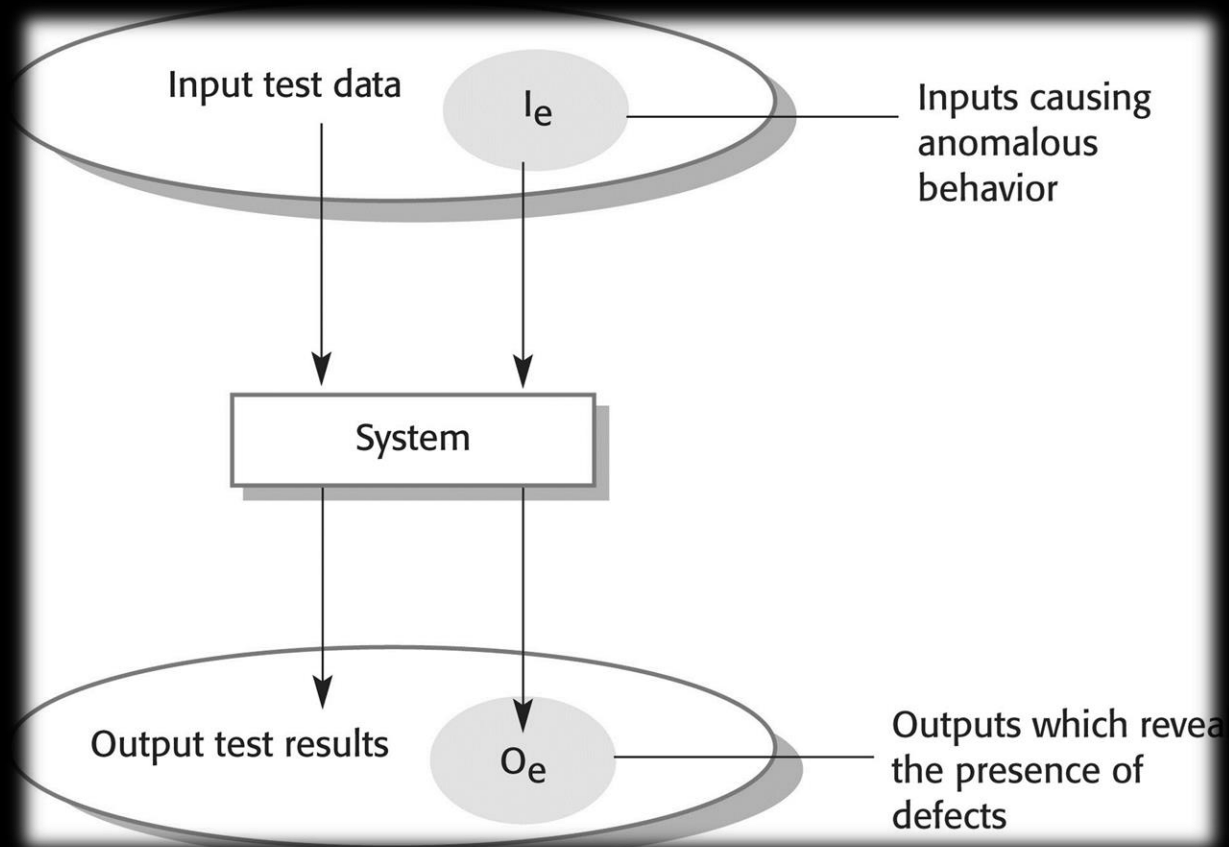
- You expect the system to perform correctly using a given set of test cases that reflect the system's expected use.
- Validation testing aims to demonstrate that the software meets the requirements
- A successful test shows that the system operates as intended.

»» The second goal leads to defect testing

- The test cases are designed to expose defects.
- Defect testing aims to discover faults in the software where its behavior is incorrect or not in conformance with its specification
- A successful test is a test that makes the system perform incorrectly and so exposes a defect in the system.



AN INPUT-OUTPUT MODEL OF PROGRAM TESTING





VERIFICATION VS VALIDATION

»» Verification:

- “Are we building the product right”.
- The software should conform to its specification.

»» Validation:

- “Are we building the right product”.
- The software should do what the user really requires.





VERIFICATION VS VALIDATION CONFIDENCE

- »» Aim of V & V is to establish confidence that the system is 'fit for purpose'.
- »» Depends on system's purpose, user expectations and marketing environment
 - Software purpose
 - The level of confidence depends on how critical the software is to an organisation.
 - User expectations
 - Users may have low expectations of certain kinds of software.
 - Marketing environment
 - Getting a product to market early may be more important than finding defects in the program.



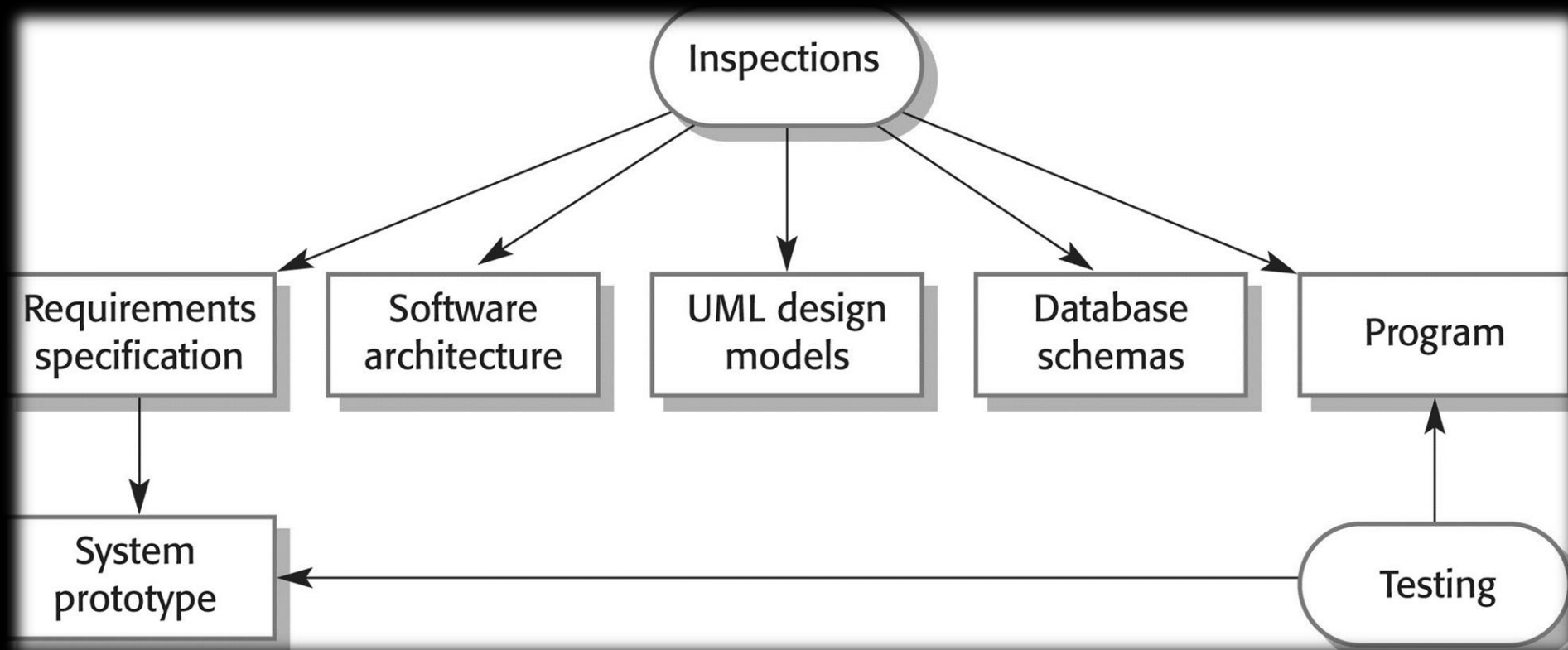


INSPECTIONS AND TESTING

- Software inspections: Concerned with analysis of the static system representation to discover problems (static verification)
- Software testing: Concerned with exercising and observing product behavior (dynamic verification)
- Inspections and testing are complementary and not opposing verification techniques.
- Both should be used during the V & V process.
- Inspections can check conformance with a specification but not conformance with the customer's real requirements.
- Inspections cannot check non-functional characteristics such as performance, usability, etc.



INSPECTIONS AND TESTING





SOFTWARE INSPECTIONS

- »» These involve people examining the source representation with the aim of discovering anomalies and defects.
- »» Inspections not require execution of a system so may be used before implementation.
- »» They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- »» They have been shown to be an effective technique for discovering program errors.





ADVANTAGES OF INSPECTIONS

- »» During testing, errors can mask (hide) other errors. Because inspection is a static process, you don't have to be concerned with interactions between errors.
- »» Incomplete versions of a system can be inspected without additional costs. If a program is incomplete, then you need to develop specialized test harnesses to test the parts that are available.
- »» As well as searching for program defects, an inspection can also consider broader quality attributes of a program, such as compliance with standards, portability and maintainability.





STAGES OF TESTING

- »» Development testing, where the system is tested during development to discover bugs and defects.
- »» Release testing, where a separate testing team test a complete version of the system before it is released to users.
- »» User testing, where users or potential users of a system test the system in their own environment.





DEVELOPMENT TESTING

- »» Development testing includes all testing activities that are carried out by the team developing the system.
 - Unit testing, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
 - Component testing, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
 - System testing, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.





UNIT TESTING

- »» Unit testing is the process of testing individual components in isolation.
- »» It is a defect testing process.
- »» Units may be:
 - Individual functions or methods within an object
 - Object classes with several attributes and methods
 - Composite components with defined interfaces used to access their functionality.





OBJECT CLASS TESTING

- »» Complete test coverage of a class involves
 - Testing all operations associated with an object
 - Setting and interrogating all object attributes
 - Exercising the object in all possible states.
- »» Inheritance makes it more difficult to design object class tests as the information to be tested is not localised.





AUTOMATED TESTING

- »» Whenever possible, unit testing should be automated so that tests are run and checked without manual intervention.
- »» In automated unit testing, you make use of a test automation framework to write and run your program tests.
- »» Unit testing frameworks provide generic test classes that you extend to create specific test cases. They can then run all of the tests that you have implemented and report, often through some GUI, on the success of otherwise of the tests.





AUTOMATED TEST COMPONENTS

- »» A setup part, where you initialize the system with the test case, namely the inputs and expected outputs.
- »» A call part, where you call the object or method to be tested.
- »» An assertion part where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful if false, then it has failed.





TESTING STRATEGIES

- Partition testing, where you identify groups of inputs that have common characteristics and should be processed in the same way.
 - You should choose tests from within each of these groups.
- Guideline-based testing, where you use testing guidelines to choose test cases.
 - These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.



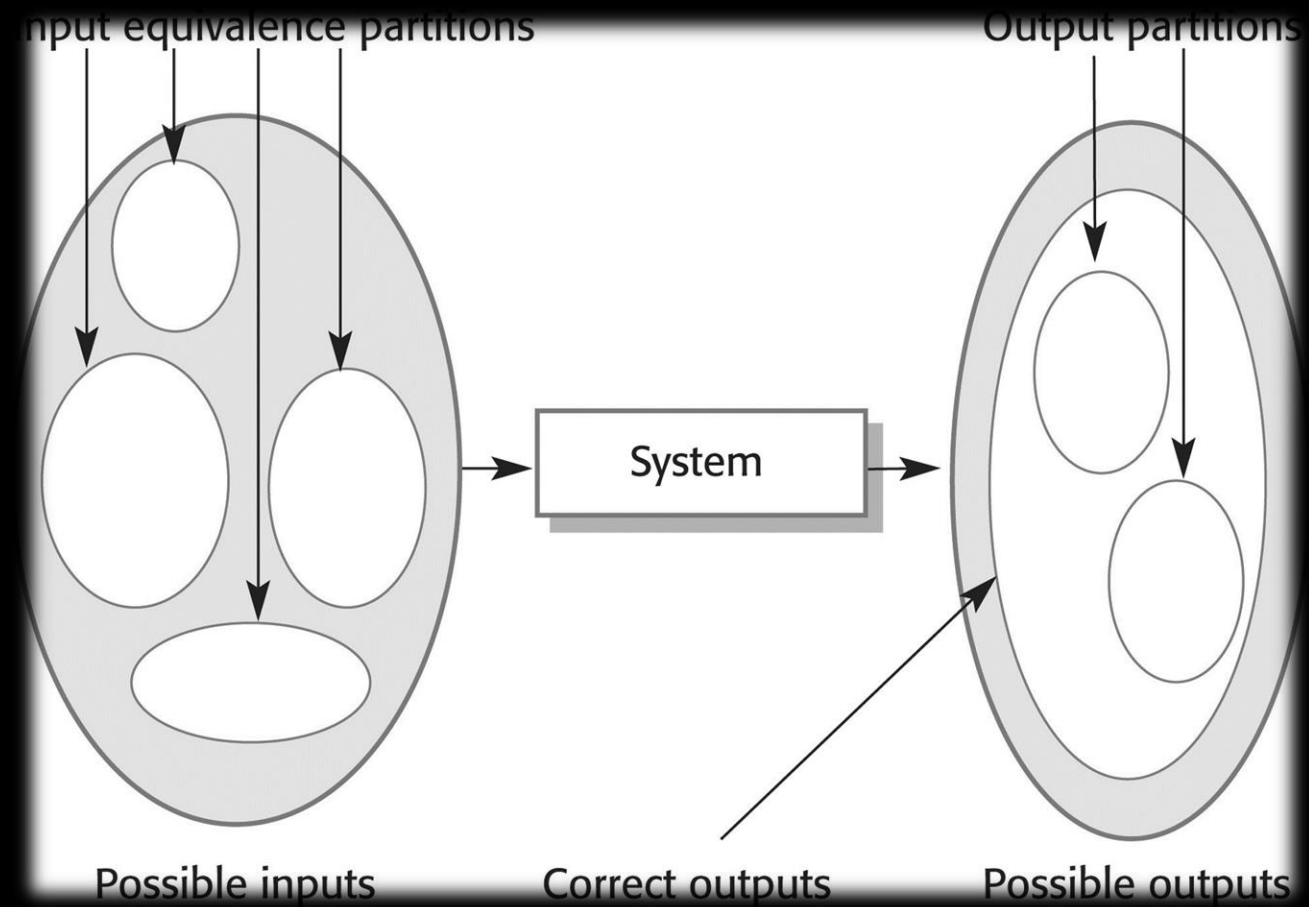


PARTITION TESTING

- »» Input data and output results often fall into different classes where all members of a class are related.
- »» Each of these classes is an equivalence partition or domain where the program behaves in an equivalent way for each class member.
- »» Test cases should be chosen from each partition.

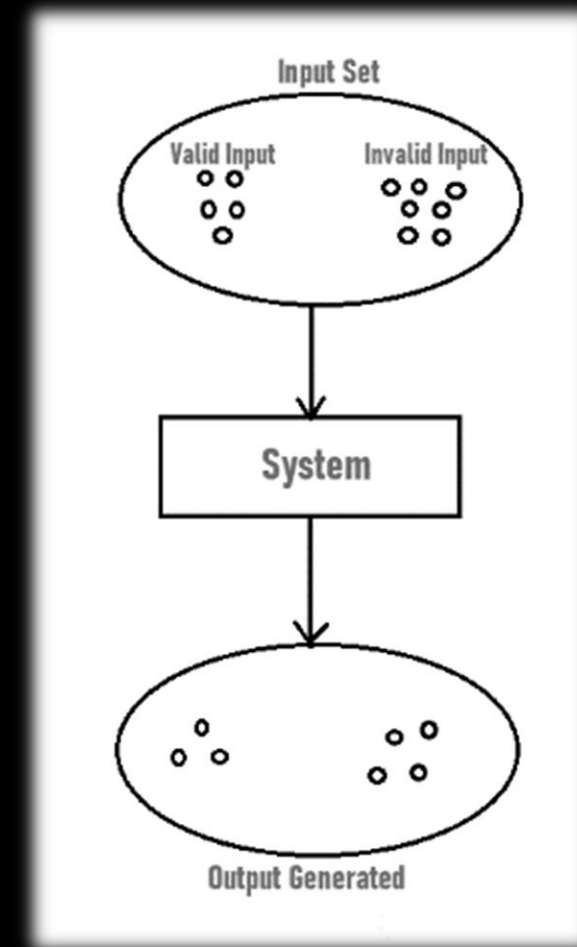


EQUIVALENCE PARTITIONING



EQUIVALENCE PARTITIONING

Equivalence Partitioning is also known as Equivalence class partitioning (ECP). It is a software testing technique or black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived.





COMPONENT TESTING

- Software components are often composite components that are made up of several interacting objects.
- You access the functionality of these objects through the defined component interface.
- Testing composite components should therefore focus on showing that the component interface behaves according to its specification.
 - You can assume that unit tests on the individual objects within the component have been completed.



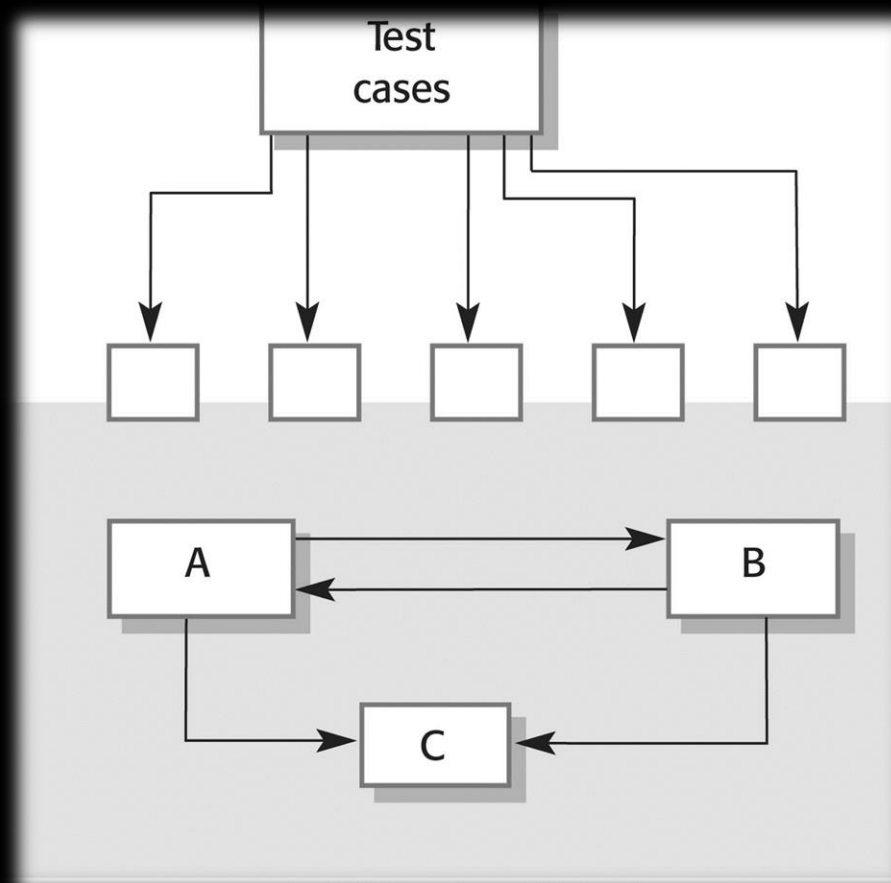


INTERFACE TESTING

- Objectives are to detect faults due to interface errors or invalid assumptions about interfaces.
- Interface types
 - Parameter interfaces Data passed from one method or procedure to another.
 - Shared memory interfaces Block of memory is shared between procedures or functions.
 - Procedural interfaces Sub-system encapsulates a set of procedures to be called by other sub-systems.
 - Message passing interfaces Sub-systems request services from other sub-systems



INTERFACE TESTING





INTERFACE ERRORS

»» Interface misuse

- A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order.

»» Interface misunderstanding

- A calling component embeds assumptions about the behaviour of the called component which are incorrect.

»» Timing errors

- The called and the calling component operate at different speeds and out-of-date information is accessed.





SYSTEM TESTING

- »» System testing during development involves integrating components to create a version of the system and then testing the integrated system.
- »» The focus in system testing is testing the interactions between components.
- »» System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.
- »» System testing tests the emergent behaviour of a system.



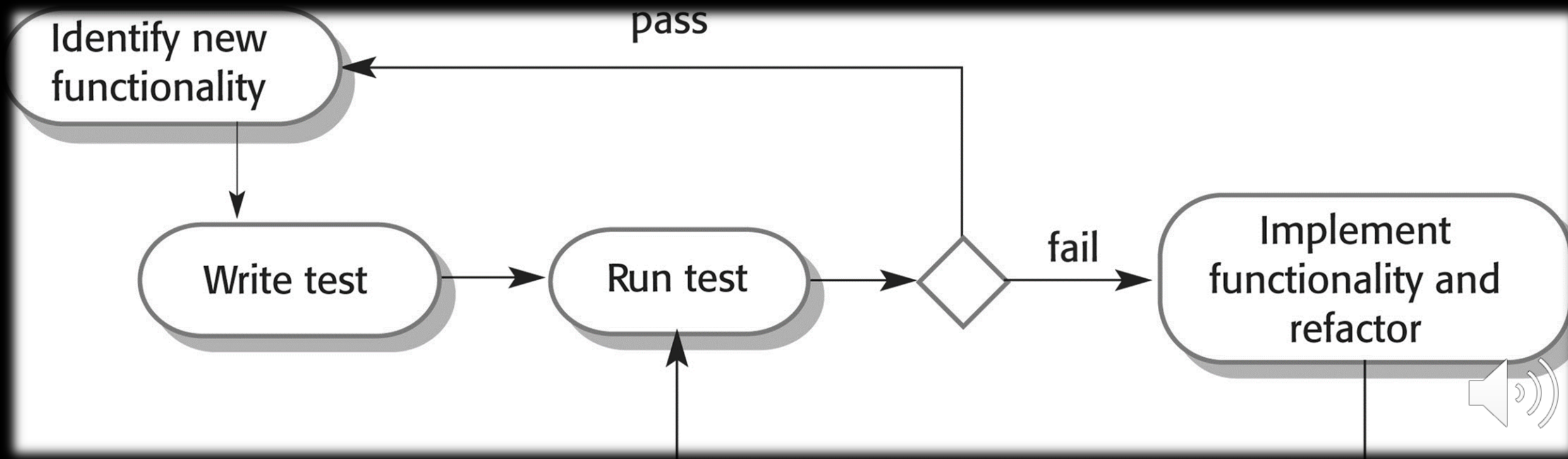


TEST-DRIVEN DEVELOPMENT

- »» Test-driven development (TDD) is an approach to program development in which you inter-leave testing and code development.
- »» Tests are written before code and 'passing' the tests is the critical driver of dev.
- »» You develop code incrementally, along with a test for that increment. You don't move on to the next increment until the code that you have developed passes its test.
- »» TDD was introduced as part of agile methods such as XP. However, it can also be used in plan-driven development processes.



TEST-DRIVEN DEVELOPMENT





TDD PROCESS ACTIVITIES

- Start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test.
- Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.





BENEFITS OF TEST-DRIVEN DEVELOPMENT

- »» Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- »» Regression testing
 - A regression test suite is developed incrementally as a program is developed.
- »» Simplified debugging
 - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.
- »» System documentation
 - The tests themselves are a form of documentation that describe what the code should be doing.





RELEASE TESTING

- »» Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.
- »» The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use.
 - Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- »» Release testing is usually a black-box testing process where tests are only derived from the system specification.





RELEASE TESTING AND SYSTEM TESTING

- »» Release testing is a form of system testing.
- »» Important differences:
 - A separate team that has not been involved in the system development, should be responsible for release testing.
 - System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).





PERFORMANCE TESTING

- »» Part of release testing may involve testing the emergent properties of a system, such as performance and reliability.
- »» Tests should reflect the profile of use of the system.
- »» Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable.
- »» Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behaviour.





USER TESTING

- »» User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- »» User testing is essential, even when comprehensive system and release testing have been carried out.
 - The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system. These cannot be replicated in a testing environment.





TYPES OF USER TESTING

»» Alpha testing

- Users of the software work with the development team to test the software at the developer's site.

»» Beta testing

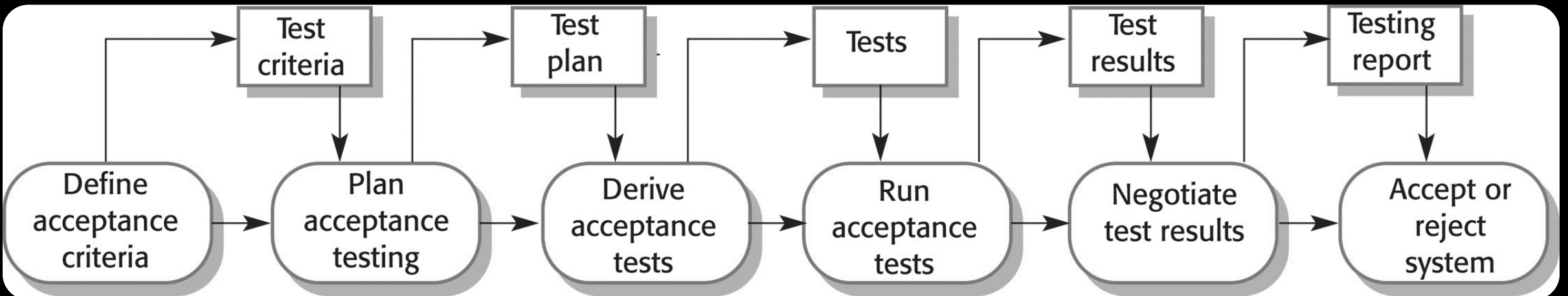
- A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.

»» Acceptance testing

- Customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment. Primarily for custom systems.



THE ACCEPTANCE TESTING PROCESS





AGILE METHODS AND ACCEPTANCE TESTING

- »» In agile methods, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.
- »» Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made.
- »» There is no separate acceptance testing process.
- »» Main problem here is whether or not the embedded user is 'typical' and can represent the interests of all system stakeholders.





SOFTWARE QUALITY MANAGEMENT

- »» SQM is a systematic process of ensuring that software meets the highest standards in terms of functionality, reliability, and user satisfaction. It is a disciplined approach to overseeing and enhancing all aspects of software development, deployment, and maintenance to ensure the final product meets or exceeds user expectations.
- »» SQM encompasses processes, standards, and practices that focus on functionality, performance, reliability, and user satisfaction, ultimately creating high-quality software products.





SOFTWARE QUALITY MANAGEMENT

- »» It breeds a process of constant improvement of product quality, integrated into the development process until the expectations are achieved. Software quality management aims to create a working environment where quality is the responsibility of everyone on the team.
- »» SQM is important because it helps to identify and correct errors in software, which is essential for the effective implementation of a project. The quality of the software is simply the measurement of how well the product has been able to fulfill your needs.





SOFTWARE QUALITY MANAGEMENT

- SQM metrics are quantitative measurements used to assess the various aspects of software development, deployment, and maintenance processes. These metrics provide objective data that help evaluate the quality of software products, processes, and teams.
- Metrics highlight the health of your product, processes, and project with actionable insights. With all the right data, you can easily:
 - Build better strategies to guide you throughout the project lifecycle.
 - Identify all the areas that need attention.
 - Make decisions driven by strategies and business information.



